

SchoolApp360 Admin Panel — Full Production Readiness Audit

Scope: Full codebase audit — Next.js App Router, plain JavaScript, PostgreSQL/Prisma, Tailwind CSS, deployed on Vercel. **Method:** Static code review only (no code changed as part of this audit). File:line citations throughout are from actual reads of the repository.

Executive Summary

Overall Score: 42/100

This codebase is unusually well-documented and internally consistent for its size — the `lib/` data-access layer, naming conventions, and per-domain constant-splitting pattern are followed disciplined and the schema is thoughtfully commented. That is real engineering quality. But it is **not production-ready**, for one dominant reason: **authentication is applied inconsistently at the API layer**, and at least ten mutating routes — several of them financial — have **no auth check at all**. Combined with a hardcoded, UI-displayed SuperAdmin password and a genuine cross-tenant authorization race condition (a `globalThis` singleton doubling as auth-adjacent state on a shared serverless deployment), this is a **Critical, launch-blocking security posture**, not a polish item.

The architecture, database schema, and code organization are good enough to build on. The security and authorization layer needs a dedicated remediation pass before any real school's data — especially fee/payment data — touches this system in production.

Category Scores

Category	Score
Architecture	6/10
Database	6/10
API Design	4/10
Security	2/10
Performance	4/10
Multi-School Readiness	3/10
Role & Permissions	3/10
UI/UX	5/10
Business Logic	4/10
Deployment Readiness	3/10
Code Quality	6/10

1. Project Architecture Audit — Score: 6/10

What's good:

- `lib/*.js` consistently exposes `getAllX / getXById / addX / updateX` data-access functions, and `page.jsx` server components call into `lib/` rather than touching Prisma directly — verified clean in `app/dashboard/students/page.jsx`, `app/dashboard/teachers/page.jsx`, `app/dashboard/exams/page.jsx`. No separation-of-concerns violation (raw Prisma in a `page/component`) was found anywhere in the sample.
- Client-safe constants are consistently split from Prisma-importing sibling files (`lib/studentConstants.js` vs `lib/students.js`, `lib/noticeConstants.js` vs `lib/notices.js`, `lib/calendarEventConstants.js` vs `lib/calendarEvents.js`, etc.) — this is a deliberate, well-executed pattern that prevents server-only code leaking into client bundles.
- 151/235 components (64%) are `'use client'`. Spot-checked 15 — the overwhelming majority genuinely need interactivity (modals, forms, filter/pagination state). This is not lazy over-marking; it's a structural consequence of the fetch-all-then-slice-client-side pattern (see Performance).

Problems:

File	Problem	Impact	Recommendation	Priority
<code>lib/students.js:100</code> , <code>lib/teachers.js:151</code>	<code>getSchoolStudents/getSchoolTeachers</code> still default to the stale <code>getActiveSchoolId()</code> singleton when no <code>schoolId</code> is passed	Currently safe (every live caller passes an explicit <code>schoolId</code>) but is a landmine for the next contributor who forgets the argument	Remove the default; make <code>schoolId</code> a required parameter	High
<code>lib/api.js</code> (1,514 lines)	Largest file in the repo, generically named — unclear if it's a fetch-wrapper library or business logic	Hard to navigate, invites more unrelated code to accrete into it	Split by domain to match the rest of <code>lib/</code> 's convention	Medium
<code>lib/classes.js:90</code>	Mixes Class/Section CRUD with direct <code>prisma.student.findMany</code> roster queries	Blurs module boundary between "classes" and "students"	Move roster queries into <code>lib/students.js</code>	Low

2. Database Audit (PostgreSQL) — Score: 6/10

Critical

- **No onDelete behavior specified from Payment / StudentFee → Student**
(prisma/schema.prisma:332 , :1035), defaulting to Restrict , while ExamMark , ExamResult , ExamOptionalEnrollment , ParentStudentLink all correctly Cascade from Student (:843 , :887 , :806 , :133). lib/students.js:342-350 's deleteStudent() does a bare prisma.student.delete() with no FK-violation handling. **Any student who has ever had a fee generated (which happens automatically per term) will 500 on delete.** This is a near-guaranteed crash the first time an admin deletes/transfers a billed student.
- **Only one migration folder exists** (prisma/migrations/20260830172048_init) despite extensive schema evolution documented in the project's own changelog. This means prisma db push has been the actual deployment mechanism, not prisma migrate — **no versioned schema history, no rollback path, no audit trail** of what changed in production and when. package.json 's build script (prisma generate && next build) never runs migrations at deploy time either.

High

- Notice and CalendarEvent models filter by schoolId on every list query (lib/notices.js:44 , lib/calendarEvents.js:30) but neither has @@index([schoolId]) — compare Homework 's @@index([schoolId, academicSession, className, sectionName]) at schema.prisma:665 . Sequential scans at scale.
- Heavy Json -blob usage for data that gets scanned in application code rather than filtered in SQL — Teacher.assignments is loaded whole and filtered in JS every request to compute class scope (lib/iam.js , lib/subjects.js:77) rather than queried relationally. Fine today, breaks the moment cross-teacher reporting ("who teaches Class 8B") is needed.
- Business invariants ("one Class Teacher per section," "one Active AcademicSession per school") are application-level only (schema.prisma:537 , :673) — no DB constraint. Concurrent saves can produce two "Active" sessions simultaneously.

Medium

- No unique constraint on Teacher.email / Student guardian email — likely intentional (schools reuse emails across siblings/staff), but confirm.

What's genuinely good: Student , Teacher , Class , Section , ExamMark , ExamSchedule , Attendance all have correctly-scoped schoolId -first composite indexes/uniques. The schema is unusually well-commented for its size — most design tradeoffs are explained inline, not silent.

3. API Audit

No shared middleware auth for /api/ .** middleware.js only attaches CORS headers to API routes and returns early — every route must self-guard via lib/iam.js 's require*() helpers, and consistency is exactly as good as each individual route remembered to do that. It isn't uniform.

Endpoint	File	Problem
POST /api/fees/collect	app/api/fees/collect/route.js	No auth guard

POST /api/fees/structures	app/api/fees/structures/route.js	No auth guard schoolId filter GET
GET/POST/DELETE /api/fees/students/[id]/discount	app/api/fees/students/[id]/discount/route.js	No auth guard
PUT /api/school/profile, /branding, /contact, /preferences, /attendance- settings	app/api/school/*/route.js	No auth guard any of these
POST/PATCH /api/sessions/**	app/api/sessions/**/route.js	No auth guard
POST /api/admins/[id]/reset-password	app/api/admins/[id]/reset-password/route.js	No auth guard
POST /api/admins/[id]/permissions, /status	app/api/admins/[id]*/route.js	No auth guard
POST /api/schools/[id]/manage	app/api/schools/[id]/manage/route.js	No auth guard before mutating the shared global "active school" state
GET /api/exams/[id]	app/api/exams/[id]/route.js:6-16	Checks a user exists but does not scope getExam by schoolId
GET /api/fees/student/[studentId]/summary	app/api/fees/student/[studentId]/summary/route.js	Ownership check only runs if (actor?.role !== 'Parent') — skipped entirely when actor is
PATCH /api/attendance/[id]/lock	app/api/attendance/[id]/lock/route.js	Role check using lib/currentUser global toggle the real session
All 117 route files	—	Zero usages lib/schemas.js

		zod schemas any API route
Students/Teachers/Classes/CalendarEvents [id] routes (sampled)	multiple	Correctly solved by resolveSchool: server-side

4. Security Audit

Critical

- 1. Hardcoded, UI-displayed SuperAdmin password.** `components/SuperAdminLoginForm.jsx:17-18` defines and *renders on the public login page* `{ email: 'superadmin@edumanage.io', password: 'SuperAdmin@123' }`. `prisma/seed.js:78,84` seeds this exact credential pair for the real platform-owner account. **The single highest-privilege account in the entire system has its password committed to source and shown to any visitor of the login screen.**
- 2. Systemic missing authentication on mutating/financial endpoints.** At minimum: `fees/collect`, `fees/structures`, `fees/students/[id]/discount`, `school/profile` + sibling settings routes, `sessions` + its `[id]` actions, `admins/[id]/reset-password`, `admins/[id]/permissions`, `admins/[id]/status`, `schools/[id]/manage` — **zero `require*()` / `getSession()` calls found in any of them.** These are directly exploitable by anyone who has the URL (trivially discoverable from the bundled client JS). Fee tampering, admin password resets, and privilege changes are all reachable with no login whatsoever.
- 3. `globalThis.__CURRENT_USER__` is shared, mutable, auth-adjacent state on a multi-tenant serverless deployment.** `lib/currentUser.js:12` — one process-wide object determines "SchoolAdmin vs Teacher" for every route still calling `getCurrentUser()` / `getCurrentActor()` directly (`app/api/attendance/route.js`, `attendance/report`, `attendance/roster`, `attendance/[id]/lock` — these four have **no real-session fallback at all**, their auth runs entirely off this global). On Vercel, warm instances are reused across concurrent requests from *different users*. `app/api/schools/[id]/manage/route.js` — itself unauthenticated (see above) — flips this exact global. **One anonymous request can change what another concurrent user's Attendance authorization resolves to.** This is a live cross-tenant authorization race, not a theoretical one.

High

- 4. No rate limiting or lockout anywhere in the codebase** (`grep -rli "rate.?limit"` → zero matches). Teacher login, Parent login, SchoolAdmin/SuperAdmin login Server Actions are all brute-forceable with unlimited attempts, no CAPTCHA, no throttling.
- 5. Access-Control-Allow-Origin: * on every /api/** route** (`middleware.js:45`), justified in-comment by "auth is Bearer-token, not cookie" — but `lib/auth/session.js` also accepts the session **cookie**, and routes like `app/api/auth/parent/switch/route.js` work purely off it. A wildcard origin combined with any cookie-authenticated route is a cross-origin risk if credentials mode is ever enabled on a fetch, now or by a future change.
- 6. Financial endpoints unauthenticated** (folded in from API Audit for severity) — an attacker can zero out or discount any student's fees, or fabricate a manual-payment record, with a single unauthenticated POST.

Medium

7. **No server-side enforcement of R2 upload size limits.** `lib/storage.js:33-40` 's `getPresignedUploadUrl` builds a `PutObjectCommand` with no `ContentLength` / `content-length-range` condition — every `*-upload-url/route.js` 's `fileSize` check is client-reported and unenforced at the storage layer. A malicious authorized user can upload arbitrarily large objects under a "2MB" labeled key.
8. **Zero input-validation-library usage across all 117 API routes** — `lib/schemas.js` exists (616 lines of zod schemas) but is imported by nothing in `app/api`.
9. Inconsistent 404-vs-403 information-leak discipline (some routes correctly hide existence via 404, others reveal it via 403) — low-severity, just inconsistent defense-in-depth.

Low

10. Plaintext demo credentials committed in `prisma/seed.js` (`admin@greenwoodhigh.edu / Welcome@123` , `info@abcpublicschool.edu / Welcome@123` , a `Parent@123` seed) and `lib/teacherSeedData.js` .
Low risk if never seeded against production — confirm this is actually true.

What checked out fine: sampled `[id]` -scoped routes for students, teachers, classes, calendar events, parent exam results, and student attendance correctly scope by `resolveSchoolId()` / `actor.studentId` server-side. The IDOR pattern is not universal — it's concentrated exactly in the routes that skip auth entirely (listed above).

5. Performance Audit — Score: 4/10

~100 unbounded `findMany()` calls across `lib/*.js` — no `take` / pagination, scaling linearly with school size:

- `lib/students.js:101` (`getSchoolStudents`) — backs the entire Students page.
- `lib/teachers.js:152` — same for Teachers.
- `lib/fees.js` — 8+ separate unbounded queries (`:78,417,431,519,688,826,1047,1076`).
- `lib/attendance.js:178` — **no date bound**, can return the entire historical attendance table.
- `lib/exams.js:311-321` , `lib/dashboard.js:304-322` — multiple unbounded queries run in `Promise.all` on every dashboard load, compounding cost.
- `lib/examMarks.js` (12 sites), `lib/examResults.js` (9 sites), `lib/notices.js:171-179` , `lib/homework.js:168-195` .

Structural "fetch-everything, slice in the browser" pattern — confirmed on the two flagship list pages, the exact same defect class as the already-fixed base64-logo bug:

- `app/dashboard/students/page.jsx:19-20` → unbounded `getAllStudents(schoolId)` → passed whole into `StudentsExplorer` → `components/students/StudentsExplorer.jsx:88` slices client-side.
- `app/dashboard/teachers/page.jsx:13-14` → same pattern via `TeachersExplorer.jsx:45` .

Every Students/Teachers page load currently ships the **entire** roster to the browser regardless of which page of 10-25 rows is displayed. Fine at 140 students (this session's test data); a measurable, then severe, problem well before 5,000.

next/image is used nowhere in the codebase. Every image — student/teacher photos, school logos, profile pictures — is a raw `<img>` tag (`components/dashboard/Sidebar.jsx:222` , `Topbar.jsx:117` , `students/StudentsTable.jsx:23` , `teachers/TeachersTable.jsx:23` , `teachers/TeacherProfileHeader.jsx:70` , `superadmin/SchoolOverviewCard.jsx:9` , `superadmin/SchoolsTable.jsx:22` , `settings/BrandingPreview.jsx:19` , `settings/LogoUploader.jsx:60` , `setup/steps/StepComplete.jsx:20`). Zero automatic resizing/format optimization/lazy-loading anywhere.

`next.config.mjs` is effectively empty — no `images.remotePatterns` , no custom headers.

6. Multi-School ERP Audit — Score: 3/10

This is the single most severe architectural finding in the whole audit.

Two `lib/*.js` modules remain pure in-memory (no Prisma import) despite everything else having been migrated to Postgres:

- `lib/currentUser.js` — `globalThis.__CURRENT_USER__`, a single object shared by every concurrent request on the same warm Node process, for **every school on the platform**. Drives real authorization in `lib/roleGuard.js`, four Attendance routes (with no real-session fallback), Homework, Notices, Exams marks-entry, and print-marksheet pages.
- `lib/school.js` — `globalThis.__ACTIVE_SCHOOL__`, a single mutable `{id}` used as the *default parameter value* on nearly every exported function in `lib/students.js` and `lib/teachers.js` (`schoolId = getActiveSchoolId()`). Every current caller passes an explicit `schoolId` (verified — no live leak today), but this is opt-in-safe, not enforced-safe: the moment a future route/page/cron omits the second argument, it silently reads/writes **whichever school a Super Admin last "stepped into" platform-wide**, not the caller's own school.

Concrete cross-tenant risk given this is one shared Vercel deployment, not one instance per school:

`app/api/schools/[id]/manage/route.js` (itself unauthenticated) can flip `ACTIVE_SCHOOL / CURRENT_USER` for the whole running instance. A request from School A's admin and a request from School B's admin can land on the same warm instance concurrently; whichever one last called this route (or the equally-unauthenticated `setCurrentRoleAction` toggle) determines what the *other* request's ambient in-memory state resolves to for the modules still reading it.

Also evaluated:

- Teachers teaching at multiple schools: no schema support (`Teacher.schoolId` is singular, not a join table) — not currently possible, would need a real redesign.
- Parents with children at multiple schools: `ParentAccount.schoolId` is singular; `ParentStudentLink` links to `Student` but the account itself is school-scoped — cross-school parent accounts are not supported.
- School-specific settings: well-supported via the real `School` row (branding, attendance deadlines, exam rank tie-mode) — this part is solid.

7. Role & Permission Audit — Score: 3/10

Roles that actually exist in code: SuperAdmin, SchoolAdmin, Teacher, Parent (`lib/iam.js`'s `require*()` family).

Student, Accountant, and generic Staff roles do not exist anywhere — no schema enum, no login route, no guard. Report this as a gap, not a matrix entry.

"Fake permissions" confirmed. `lib/adminConstants.js`'s `ADMIN_PERMISSIONS` (`manageStudents`, `manageTeachers`, `manageClasses`, `manageAcademicSessions`, `manageSchoolSettings`) is written by `app/api/admins/[id]/permissions/route.js` (itself unauthenticated), displayed in Settings — and **read by nothing else in the entire codebase**. A SchoolAdmin whose permissions were restricted in the UI can still do everything a full admin can via direct API calls. The permissions system is decorative.

Role x Module matrix (as actually enforced in code, not as intended):

Role \ Module	Students/Teachers	Attendance	Fees (collect/structure)	Exam Marks	Homework/Notice
SuperAdmin	Full	Partial (global-toggle dependent)	None enforced	Partial	Full
SchoolAdmin	Full	Partial	None enforced	Partial	Full
Teacher	Correctly blocked	Partial (own classes)	N/A	Partial (own schedules)	Correctly scoped
Parent	None	None	Partial (null-actor bypass — see API Audit)	None	Read-scoped to own child

Other findings: the "step into a school" flow has zero auth guard and no audit log — any caller can POST a school id and become its effective admin, with nothing recorded about who did it or when.

8. UI/UX Audit (code-based structural review — not a visual walkthrough)

Top 15 concrete improvements:

1. Add `loading.jsx` to Students, Teachers, Fees, Exams, Attendance, and the Dashboard home — the highest-traffic routes currently have **no** route-level loading state (only 7 routes in the whole app do: `classes`, `classes/[id]`, `classes/[id]/sections/[sectionId]`, `homework`, `notices`, `sessions`, `sessions/[id]`).
2. Add `error.jsx` boundaries — **zero exist anywhere in app/**. Any server-component throw currently falls through to Next's generic default error page, on every single route.
3. Add `aria-label` to icon-only buttons — only one component in the entire 235-component tree uses `aria-label`. `components/Dropdown.jsx:136,186,201` and equivalents across the tree are effectively unlabeled for screen readers.
4. Add a real focus trap to `components/Modal.jsx` — Escape-to-close exists (:21) but Tab can cycle a keyboard user out into background content while a modal is open.
5. Audit mobile responsiveness on `TeachersExplorer.jsx` and `StudentFeeDetailPanel.jsx` — zero `md:` / `sm:` / `lg:` breakpoint classes found in the grep sample; `StudentsExplorer.jsx` had only one. These are likely to overflow or misrender on phone widths.
6. Empty-state coverage ("No ... found" patterns) appears in only 21 of 235 components — likely primary list pages only; nested/secondary views (empty exam schedule, empty fee structure) weren't confirmed covered. 7-15. (Structural, not exhaustive — a real UI pass needs an actual visual/browser walkthrough, which this audit did not do): confirm loading/empty/error states are visually consistent across the dashboard, super-admin, and parent portal design languages (three different shells exist — `DashboardShell`, `SuperAdminTopbar / SuperAdminNav`, `ParentShell` — verify they don't drift stylistically); confirm table usability (sort/filter/pagination) is consistent across

Students/Teachers/Fees/Exams tables, since some were built at different points this session with slightly different UI kits; confirm every form has inline validation messaging, not just a top-of-form error banner; confirm color-contrast on the gradient-heavy buttons meets WCAG AA (not checked here — needs a visual/automated contrast tool).

Caveat: this section is based on reading component code for structural signals (presence/absence of loading/error/aria patterns), not an actual rendered walkthrough of every screen. A real UI/UX pass should be done in-browser.

9. School ERP Business Logic Audit — Score: 4/10

Ranked by real-world severity (money/grades/data-integrity first):

- Deleting a student with billing history will crash, and if that's ever worked around, destroys legally-required financial records.** `StudentFee / Payment onDelete: Cascade from Student` (`schema.prisma` ~843, ~887, ~989) + `StudentStatus` enum having only `Active / Inactive` (no `TransferredOut / Graduated / Alumni`) means there's no soft-delete path — a mid-year transfer either can't be deleted (FK error) or, if someone patches around it, silently wipes payment history schools are required to retain.
- Refunds do not exist.** `applyFeeDiscount` (`lib/fees.js:595-618`) explicitly tells the caller to "process a refund first" when a discount would drop below `paidAmount` — but no refund function exists anywhere in `lib/fees.js` . A school that overcharges or needs to refund a withdrawn student has no code path.
- Late fees are configured but never charged.** `FeeStructure.lateFee*` fields are stored and displayed (`lib/fees.js:26-32, 205-214`) but never factored into any payable-amount calculation (`lib/fees.js:451, 533`). A school that configures "₹50/day after 5-day grace" gets a UI showing that config and a parent who is simply never charged it.
- Absent-then-retake exam results always start from a forced 0.** `lib/examResults.js:182-197` sets `obtained = 0` and `allSubjectsPass = false` unconditionally for `isAbsent` , before `retakeResultPolicy` (`Best / Average / Latest`) merge logic runs — a medically-absent student who retakes has no way for that absence to be excluded from an "Average" calculation.
- No admissions/enrollment workflow exists at all.** "Add Student" is a direct record creation (`lib/students.js`) with only an admission-number-uniqueness check — no application, approval, or waitlist stage. Any school with capacity limits or a formal admissions process cannot track prospective applicants before they're already enrolled.
- Attendance unlock has no re-check of the deadline/grace window.** `app/api/attendance/[id]/lock/route.js` only checks the caller's role, never re-validates `attendanceDeadlineTime / attendanceEditGraceMinutes` on unlock — once unlocked, edits can happen arbitrarily far past the original deadline with no re-lock enforcement.
- Push/parent notifications are fire-and-forget with no delivery guarantee.** `lib/pushTokens.js` stores device tokens, but a failed push (expired token, network blip) has no retry or dead-letter handling — a parent can silently miss a notice with zero visibility to the admin that delivery failed.

10. Code Quality Audit — Score: 6/10

Files needing refactor (top 10, largest/most complex):

File	Lines	Issue
<code>lib/api.js</code>	1,514	Largest file, generically named — split by domain

lib/fees.js	1,118	20 findMany call sites; structures/payments/reminders/receipts all in one module
lib/studentSeedData.js	816	Fixture data living in lib/ alongside production logic — move to scripts//prisma/seed/
lib/schemas.js	616	Monolithic validation file, unlike every other domain's per-file split convention
lib/teachers.js	523	Structurally near-duplicate of lib/students.js (see below)
lib/examMarks.js	508	12 findMany calls, dense nested state-machine logic
lib/classes.js	479	Mixes Class/Section CRUD with student-roster queries
lib/exams.js	440	Dashboard aggregation logic embedded in general CRUD file
lib/attendance.js	407	Marking and reporting logic interleaved in one file
lib/examResults.js	379	Grade computation, publishing, querying all interleaved

Real duplication found: lib/students.js:100-124 and lib/teachers.js:151-172 are structurally near-identical (getSchoolX(schoolId = getActiveSchoolId()) → stats → getAllX). Extracting a shared makeEntityStats() helper would both remove the duplication and kill both stale getActiveSchoolId() fallbacks in one move.

Dead code: genuinely clean — zero TODO / FIXME / HACK / XXX markers found anywhere in lib/ , app/ , or components/ , and no significant commented-out code blocks. This is unusual and a real positive signal for a codebase this size.

11. Deployment Audit — Score: 3/10

- vercel.json 's regions: ["syd1"] pin is very likely silently ignored — custom function region pinning requires a Vercel Pro plan; if this project is on the Hobby tier (evidence from this session's own testing strongly suggests it is), functions run in Vercel's default region regardless of this setting.
- .env 's JWT_SECRET carries an explicit self-flagged comment: "Demo-only secret... rotate before this ever ships." If this literal value is still the one set in Vercel's production environment variables, every session token in the system is forgeable by anyone who has seen the repository.
- **No migration execution in the deploy pipeline.** Build script is prisma generate && next build only — no prisma migrate deploy . Combined with the single-migration-folder finding (Database Audit), schema changes reaching production are not gated by anything automated, and there's no rollback path for a bad schema change.
- **No security headers configured** — next.config.mjs has no headers() function; no X-Frame-Options , Content-Security-Policy , or Strict-Transport-Security . Given the app stores Aadhaar numbers and identity documents (Student.aadhaarNumber , documents Json blobs), this is a real gap for a PII-heavy admin panel.
- **SMTP is Gmail/dev-tier** per .env comments — fine for testing, a real blocker for production email volume/deliverability (password resets, fee receipts).
- No monitoring/alerting setup found beyond Vercel's own default dashboards (confirmed via this session's own use of Vercel's Observability tab) — no error-tracking service (Sentry etc.) wired in.

- No documented backup strategy for the Postgres database beyond whatever Supabase's own tier provides by default.

12. Production Readiness Audit

School size	Verdict	Notes
100 students	OK	Current unbounded-query pattern has negligible impact at this scale; this is roughly what was tested this session.
500 students	OK, with early symptoms	Students/Teachers list pages start shipping a noticeably larger payload to the browser on every load (full-roster-then-client-slice pattern). Still functional.
1,000 students	Degraded	Dashboard's multiple unbounded <code>Promise.all</code> queries (<code>lib/dashboard.js:304-322</code> , <code>lib/exams.js:311-321</code>) and the fees module's 8+ unbounded queries start to meaningfully slow page loads; Vercel serverless function duration/memory becomes a real risk on the heavier pages.
5,000 students	Not viable without fixing pagination	The fetch-all-then-slice pattern on Students/Teachers, plus unbounded attendance history queries (<code>lib/attendance.js:178</code> , no date bound), will regularly exceed comfortable serverless response times and payload sizes.
10,000 students	Not viable	Same issues, compounded; additionally the single-shared-instance <code>ACTIVE_SCHOOL/CURRENT_USER</code> global-state risk becomes proportionally more dangerous as concurrent multi-school traffic increases.

Critical Issues (launch blockers)

1. Hardcoded SuperAdmin password displayed on the public login page (`components/SuperAdminLoginForm.jsx:17-18` , `prisma/seed.js:78,84`).
2. At least 10 mutating API routes — several financial — with **zero authentication** (`fees/collect` , `fees/structures` , `fees/students/[id]/discount` , `school/profile` + settings siblings, `sessions/**` , `admins/[id]/reset-password` , `admins/[id]/permissions` , `admins/[id]/status` , `schools/[id]/manage`).
3. `globalThis.__CURRENT_USER__` / `globalThis.__ACTIVE_SCHOOL__` — shared mutable state driving real authorization decisions on a multi-tenant serverless deployment; a genuine cross-tenant race condition, not theoretical.
4. `Payment` / `StudentFee` cascade-delete-incompatible relation to `Student` with no soft-delete path — deleting a billed student either crashes or destroys required financial records.
5. `JWT_SECRET` explicitly flagged in code comments as a demo value that must be rotated before shipping — status unconfirmed for the live Vercel deployment.

High Priority Issues

6. No rate limiting/lockout on any login endpoint.
7. `Access-Control-Allow-Origin: *` on all `/api/**` combined with cookie-based auth on some routes.

8. `admins/[id]/permissions` fully unauthenticated, and the permissions it sets are read by nothing (decorative permission system).
9. ~100 unbounded `findMany()` calls across `lib/*.js` ; structural fetch-all-then-slice-client-side pattern on Students/Teachers pages.
10. `next/image` unused everywhere — every photo/logo is an unoptimized raw `<img>` .
11. Only one Prisma migration ever recorded — no schema version history or rollback path.
12. No refund workflow; late fees configured but never actually charged.

Medium Priority Issues

13. No server-side R2 upload size enforcement (client-value-only).
14. Zero zod-schema usage across all 117 API routes despite `lib/schemas.js` existing.
15. Missing `@@index([schoolId])` on `Notice / CalendarEvent` .
16. No `error.jsx` anywhere in `app/` ; sparse `loading.jsx` coverage (7 of dozens of routes).
17. Near-zero `aria-label` usage; no focus trap in `Modal.jsx` .
18. No admissions/enrollment workflow — direct-add only.
19. Absent-student exam scoring forces a 0 before retake-policy merge logic runs.
20. Attendance unlock doesn't re-check the deadline/grace window.

Low Priority Improvements

- Plaintext demo credentials in `prisma/seed.js` / `lib/teacherSeedData.js` — confirm never seeded to production.
- `lib/api.js` , `lib/fees.js` , `lib/schemas.js` are oversized and would benefit from a domain split.
- Real duplication between `lib/students.js` and `lib/teachers.js` 's stats functions.
- Weak mobile-responsive coverage on `TeachersExplorer.jsx` / `StudentFeeDetailPanel.jsx` .
- Push notifications are fire-and-forget with no retry/visibility on failure.
- No error-tracking/monitoring service wired in beyond Vercel's own dashboard.

Top 20 Action Items (sorted by impact)

1. Remove the hardcoded/displayed SuperAdmin password; rotate the real credential.
2. Add `requireSchoolAdmin()` / `requireSuperAdmin()` to every unauthenticated mutating route listed under Critical Issue #2.
3. Rotate `JWT_SECRET` in production and confirm it's no longer the demo value.
4. Eliminate `globalThis.__CURRENT_USER__` / `__ACTIVE_SCHOOL__` as authorization-relevant state — route every remaining consumer through the real session (`getCurrentUserInfo()` / `resolveSchoolId()`), no in-memory fallback for anything security-relevant.
5. Add a soft-delete/status path (`TransferredOut` / `Graduated` / `Alumni`) for `Student` / `Teacher` instead of hard delete; fix or remove the `StudentFee` / `Payment` cascade-delete crash.
6. Add rate limiting to all login endpoints.
7. Restrict `Access-Control-Allow-Origin` or fully separate cookie-auth routes from wildcard-CORS routes.
8. Either enforce `ADMIN_PERMISSIONS` in every route that checks `requireSchoolAdmin()` , or remove the permissions UI so it stops lying to admins about what it does.
9. Paginate `lib/students.js` , `lib/teachers.js` , `lib/fees.js` , `lib/attendance.js` at the query layer; stop shipping full rosters to the browser.
10. Switch to `prisma migrate` with a real migration history and add `prisma migrate deploy` to the build step.

11. Add server-side R2 upload size enforcement.
 12. Add zod validation (already-written schemas in `lib/schemas.js`) to every mutating API route.
 13. Fix the `fees/student/[studentId]/summary` null-actor ownership bypass.
 14. Build a refund workflow; wire late-fee config into actual payable calculations, or remove the config UI.
 15. Add `error.jsx` to `app/` (at minimum the root layout) and `loading.jsx` to `Students/Teachers/Fees/Exams/Attendance/Dashboard`.
 16. Add security headers (`next.config.mjs headers()` — CSP, X-Frame-Options, HSTS).
 17. Add `@@index([schoolId])` to `Notice / CalendarEvent`.
 18. Fix the absent-student-forces-0-before-retake-merge exam scoring order.
 19. Re-validate the attendance deadline/grace window on unlock, not just on mark-creation.
 20. Migrate every remaining `<img>` to `next/image`.
-

Launch Readiness Verdict: NOT READY

Evidence: A production School ERP holding student PII (including Aadhaar numbers), attendance, grades, and fee/payment records currently has (a) a hardcoded platform-owner password visible on its public login page, (b) at least ten unauthenticated endpoints capable of mutating financial and administrative data, and (c) a genuine cross-tenant authorization race condition built into shared in-memory server state on a multi-tenant serverless deployment. Any one of these three is disqualifying on its own for a launch handling real schools' data; together they represent a systemic gap in how auth was applied across the API surface, not a handful of isolated bugs. The architecture, schema design, and code organization underneath are solid enough that this is fixable in a focused remediation pass — it is not a rebuild — but it is not safe to onboard a real paying school today.

Phased Roadmap

Phase 1 — Critical Fixes (before any real school's data touches this system)

- Remove/rotate the hardcoded SuperAdmin credential and `JWT_SECRET`.
- Add auth guards to every unauthenticated mutating route (Critical Issue #2).
- Eliminate `globalThis` state from every security-relevant decision path.
- Fix the `Student / Payment` cascade-delete crash with a soft-delete status.
- Fix the fee-summary null-actor IDOR.

Phase 2 — Performance & Security

- Add rate limiting to login endpoints.
- Restrict CORS; audit every cookie-authenticated route against it.
- Add zod validation to all mutating routes.
- Paginate the unbounded queries in `students.js` / `teachers.js` / `fees.js` / `attendance.js`.
- Add security headers; add `error.jsx` / broaden `loading.jsx` coverage.
- Enforce (or remove) the `ADMIN_PERMISSIONS` system.

Phase 3 — Scalability

- Switch to `prisma migrate deploy` with real migration history.
- Move Students/Teachers/Fees list pages to server-side pagination instead of client-side slicing.
- Add missing composite indexes (`Notice`, `CalendarEvent`).
- Migrate remaining `<img>` tags to `next/image`.
- Build the refund workflow; wire late fees into actual calculations.

Phase 4 — Enterprise Readiness

- Real admissions/enrollment workflow (application → approval → enrollment).
- Multi-school teacher/parent account support (schema redesign for cross-school linkage).
- Monitoring/error-tracking service integration; documented backup/restore strategy.
- Accessibility pass (aria-labels, focus traps, contrast audit).
- Push-notification delivery guarantees (retry/dead-letter handling).